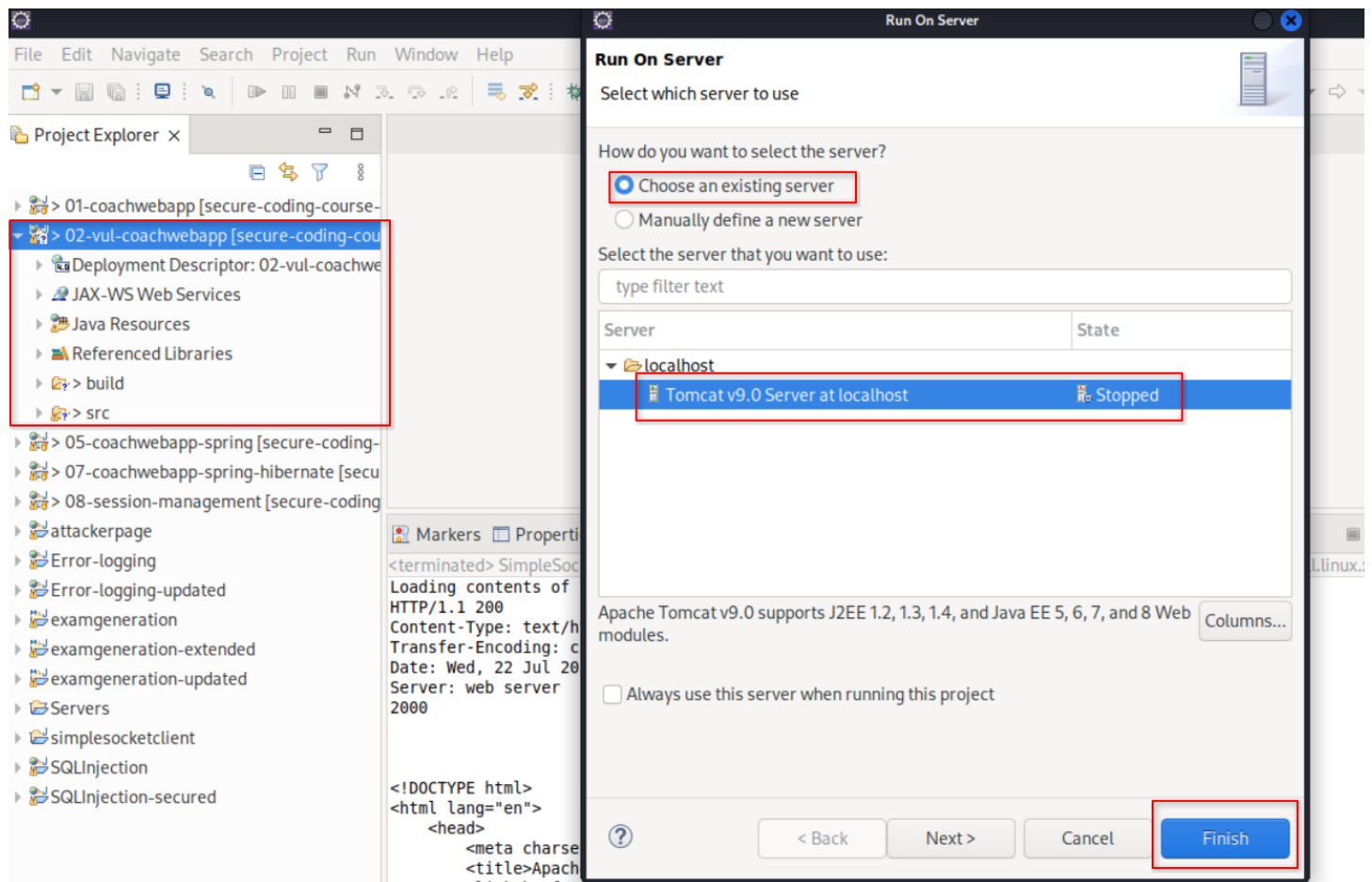


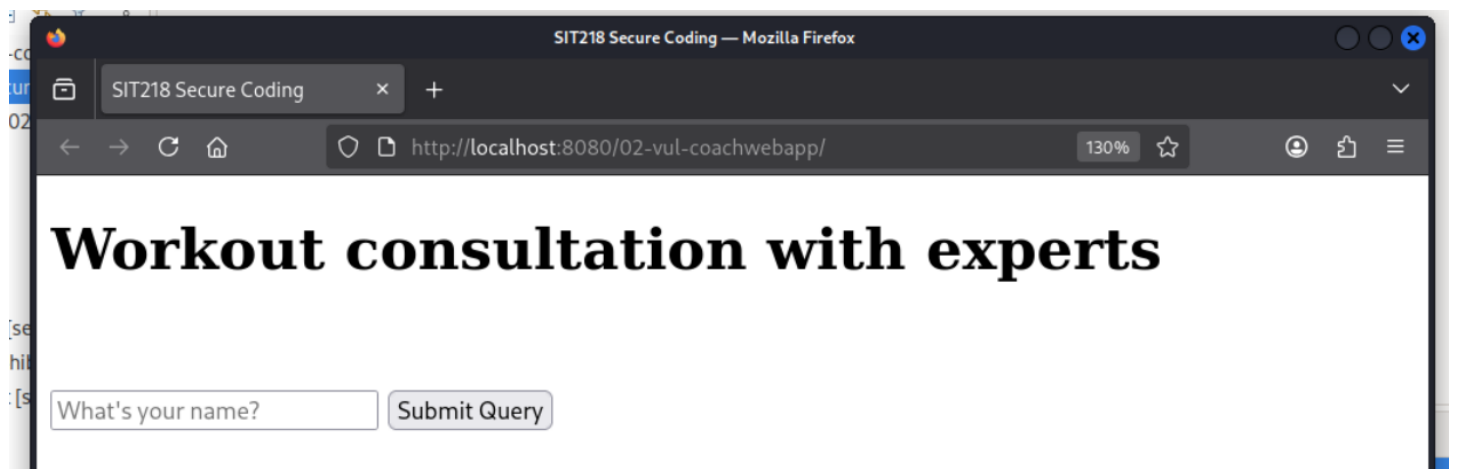
Task 3.1P

Task 1

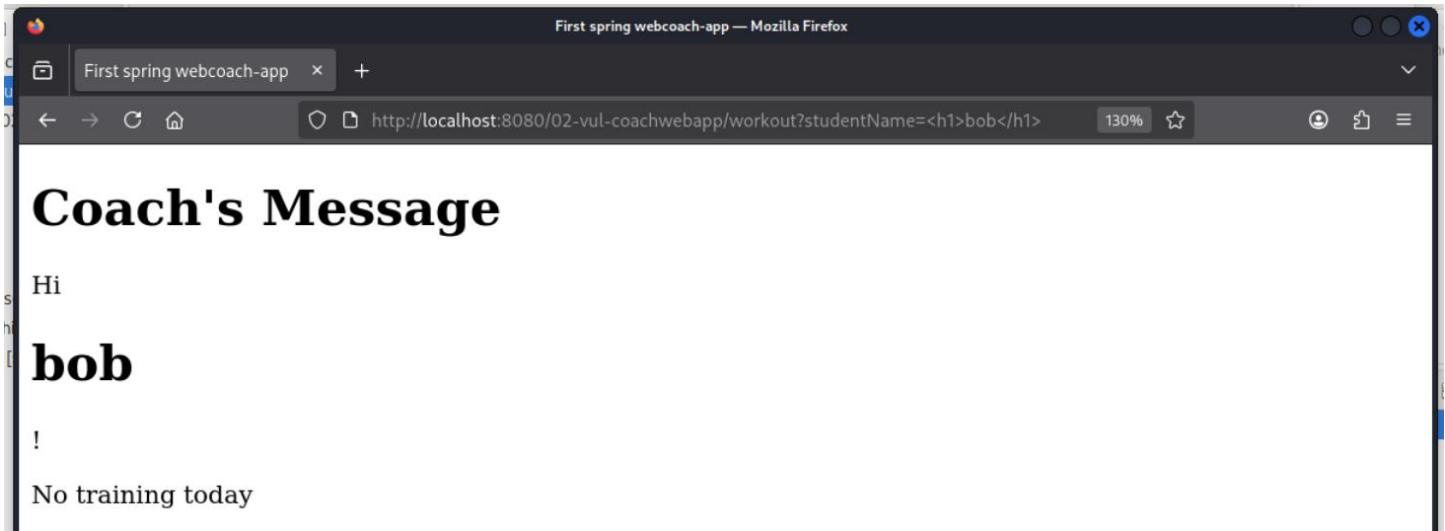
An image of Eclipse with the web server running. Since I already had the tomcat.service systemd service running and using those ports, I had to stop it and start this web app again for it to work.



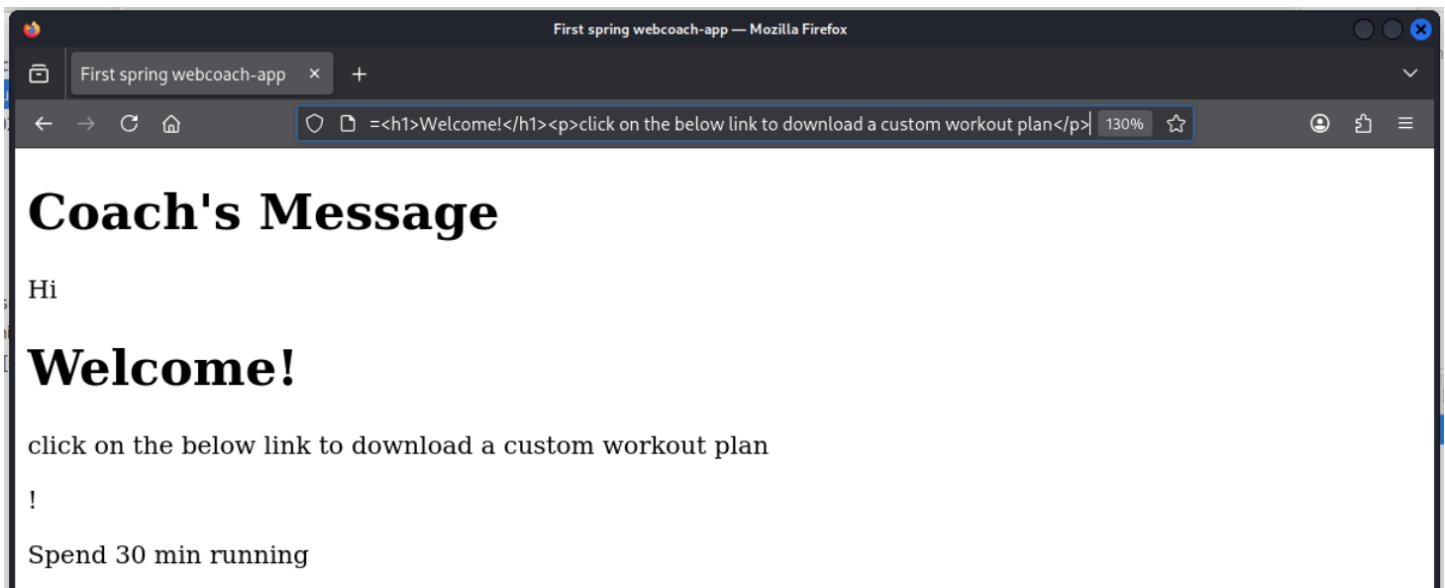
This is what a normal response looks like.



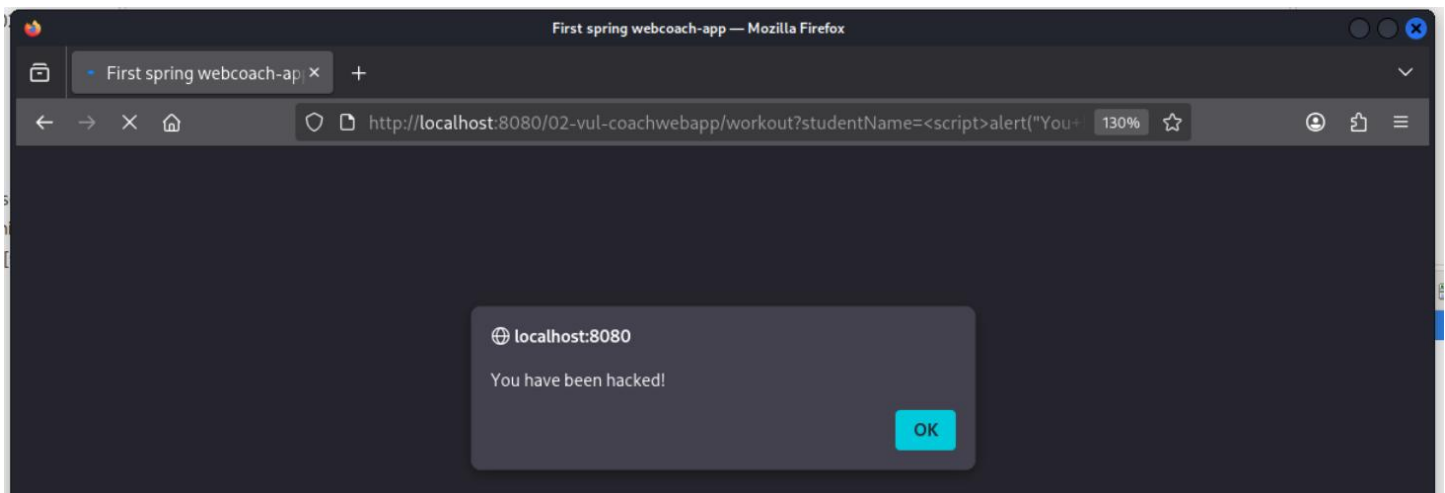
Response showing “bob” has a level one heading. This proves that the HTML is not sanitized before it gets injected to the DOM.



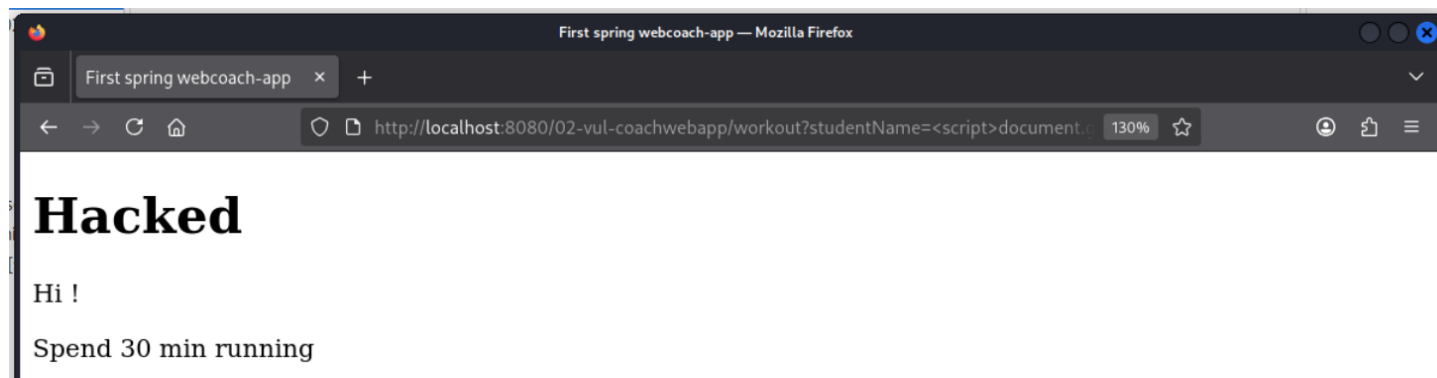
Testing out a multi-element payload.



Executing javascript code (an alert() box).



Updating elements in the DOM using JavaScript:



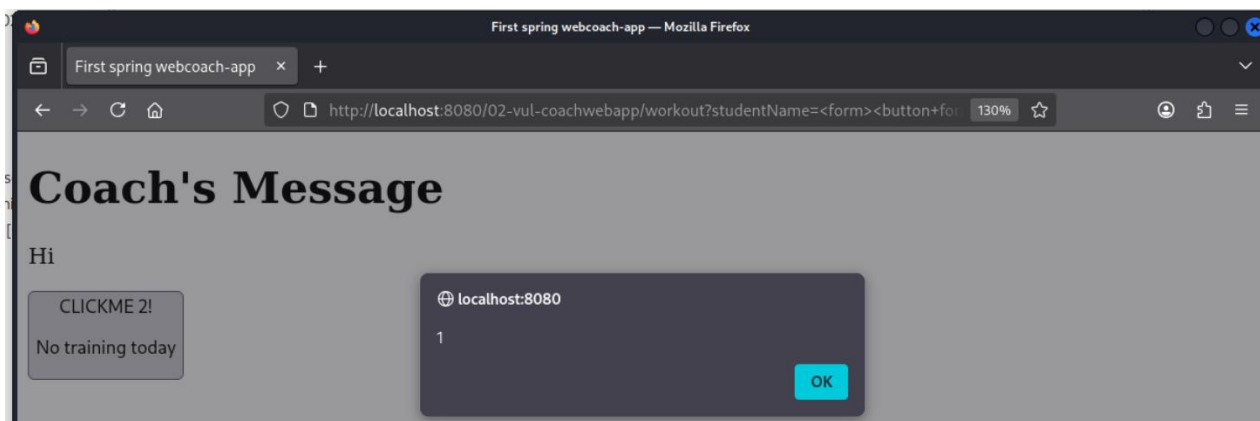
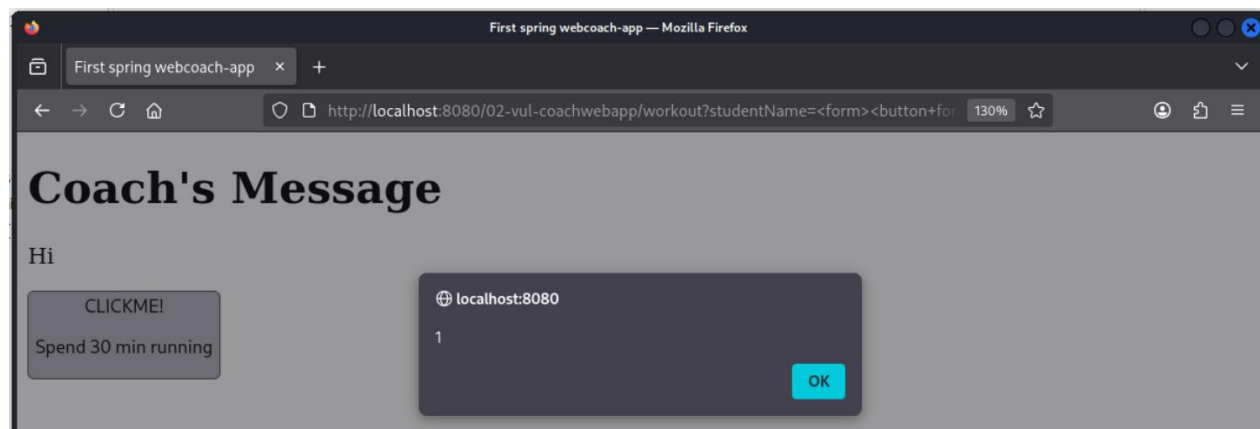
Question 5

```
String htmlResponse = "<p>" + comment + "</p>"; // Vulnerable line
```

This is a line of code written in Java. It's vulnerable to XSS because whatever the value in comment is directly inserted into the HTML without escaping it or sanitizing it. The users' input might have `<script>` tags or any other tags executing malicious javascript, thus allowing arbitrary code execution in the browser. Judging by the entire code snippet, this seems to be a reflected XSS vulnerability.

Task 2

Screenshots from both the payloads:

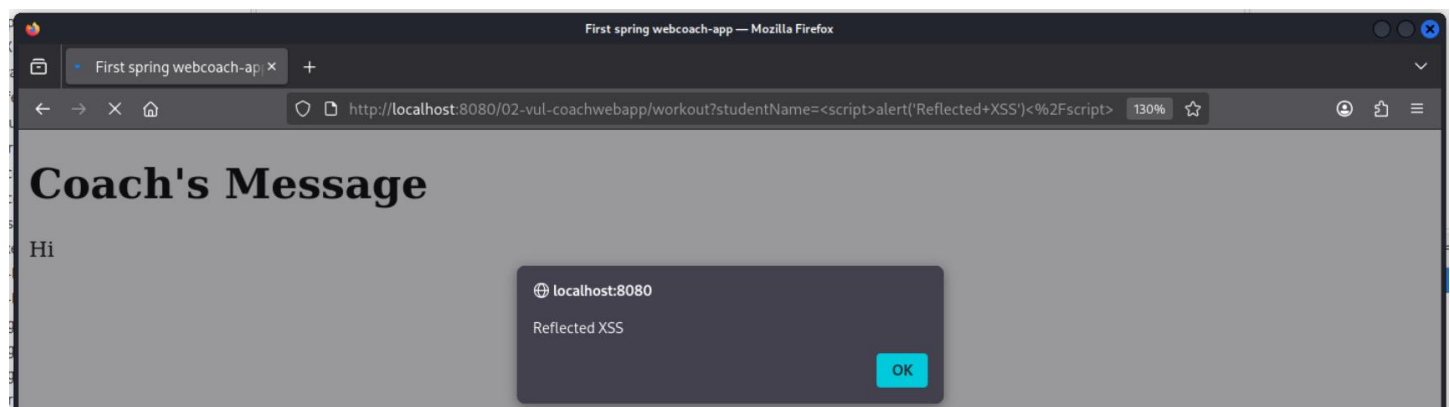


This doesn't use the `<script>` tag at all. If there is a filter in a webapp to either block or remove the `<script>` tag, our XSS payload will not get detected. Due to the unsafe approach used to deal with the users' input, the browser will still read the `<form>` and `<button>` tags as real HTML elements. The onclick and formaction attributes/events usually contain javascript in more modern web development standards. However, you could also explicitly say ``javascript:`` and have JS there as a URI. No matter what you do, the browser will execute that javascript whenever the button is clicked.

This shows why XSS protection should properly escape or encode all special characters in HTML instead of only blocking specific tags.

Another popular POC payload used is `<img src="" onerror="alert(1)" />`. This is an image tag trying to load the ``src`` image, but since it has not been properly set, the onerror trigger will execute the javascript mentioned there.

Task 3



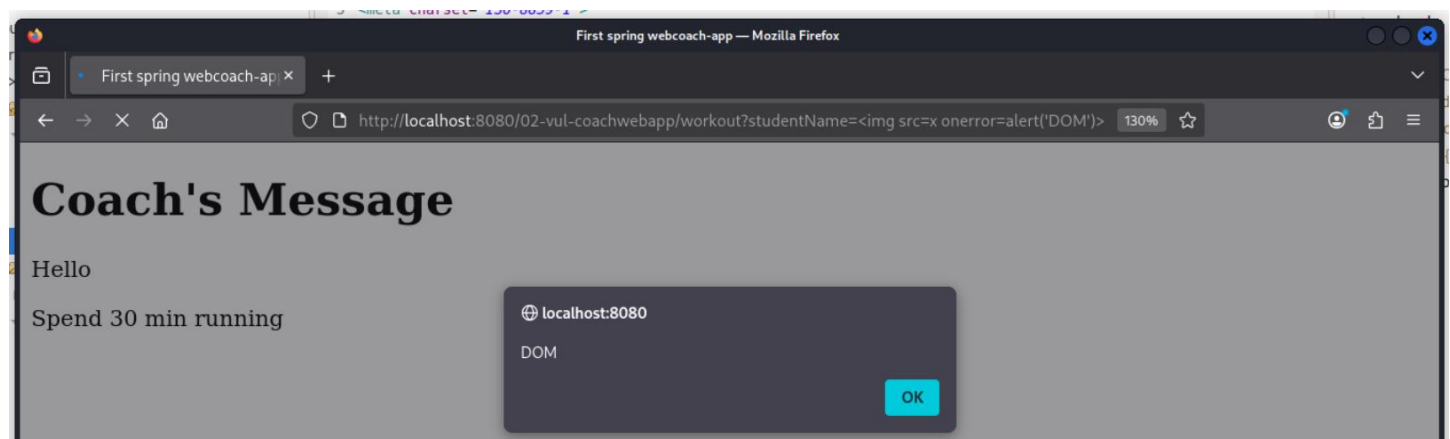
a. Note whether the alert executes and explain why

Yes, the server sends the raw input directly back in the HTML response without encoding it. The browser then reads and interprets it as a real `<script>` tag and executed the JS.

b. Did the processing of your input occur on the server side or client (browser) side?

The server simply processes the input and places it into the response without escaping it. The browser then executes the JS after reading the response. This is called a reflected XSS attack – here, the server directly reflects the users input straight back into the page.

Task 4



a. What differences did you observe between the first and second injection attempts?

The first relied on the server injecting the payload into the page but we disabled it. Now, it completely bypasses the server because the client side JS reads the payload directly from the URL.

b. What was the source of the input that triggered the alert?

It comes from the URL query string named "studentName" and we access it using the URLSearchParams javascript method and it's very widely compatible^[1].

c. Was your input processed on the server side or in the browser (client side)?

It's now being processed entirely on the client side.

d. In the case of DOM-based XSS, was the injected script reflected back to the client side?

No. Nothing is sent to the server to be reflected back. The script is taken from the URL and inserted into the DOM directly by the browser, all in the client side.

e. Compare the location of the vulnerability in both the reflected and DOM-based XSS attempts. Did it occur in the CoachController class (processed on the server) or in the workout.jsp page (executed in the client browser)?

The reflected XSS vulnerability was caused in the CoachController.java file on the server side. The DOM based XSS vulnerability is in the embedded JS inside workout.jsp on the client side.

f. Can static HTML pages be vulnerable to DOM-based XSS attacks? Provide a brief explanation.

Yep, even a completely static page can have DOM XSS if its JS reads attack controlled data like a URL, a hash or referral and inserts it into the DOM without properly sanitizing or escaping it. No server side code is required at all for this.

Task 5

Introduction

XSS is a code injection vulnerability where an attacker places malicious scripts into the content that is later loaded into another user's web browser and executed^[2]. Unlike other injection attacks like SQL injection, which mainly target the database server (or other injection attacks which targets the server), XSS targets the users browser and how the browser trusts the content sent from the website and executes them locally – allowing the attacker to execute arbitrary code JavaScript code execution within the context of that website^[3]. In most cases, the root cause is that the input entered by a user is inserted into a web page without being properly sanitized, validated or escaped. As a result, the browser treats the attacker's script as a legitimate page content and executes it^[2]. As many web browsers accept and display user input, XSS continues to be one of the most commonly reported web application vulnerabilities. It's formally classified as CWE-79 and is also documented as the attack pattern CAPEC-63^{[3][4]}.

Categories

XSS attacks can be categorized into three main categories based on where the payload is stored and how it reaches and executes on the victims browser.

Reflected XSS (non persistent): This happens when the server takes input directly from the current HTTP request, usually from a URL parameter, and immediately includes it in the response without proper sanitization or encoding^[5]. Because the payload is not stored by the application itself, the attacker needs to social engineer and convince the user to open a crafted link with the payload in it. The vulnerable coach app demonstrates this actually. A name supplied through the query string is injected directly into the page's DOM, meaning that a script placed inside the URL executes as soon as the page loads.

Stored XSS (persistent): This is when malicious input from the attacker is saved by the application, like in the database, a comment field, message log, user profile or anything else that is displayed to other users^[2]. This is generally considered more dangerous than reflected XSS because the attacker only needs to inject the payload once and once it's in the system, every user who later views this infected content / payload may execute the script, including privileged users like administrators^[2].

DOM based XSS: This differs from stored XSS because the vulnerability exists entirely within the client side JavaScript. The application reads data from a source that's controlled and influenced by the attacker, like the document.referrer, or a URL fragment and passes it into the unsafe DOM sink like innerHTML or document.write^{[2][6]}. The malicious value doesn't necessarily need to be processed or returned by the server because the server may never receive the payload in a form that it can inspect. Therefore, this can be harder to identify using traditional server side techniques – and it requires developers to analyze the applications client side JS logic^[6].

Effects

A successful XSS attack lets the injected script to run with the same permissions as the original legitimate website in the victims browser. This can allow an attacker to steal session cookies / authentication tokens, to potentially hijack account sessions and log in (without requiring a password). It may also be used to redirect users to phishing sites or other websites hosting malware, modify or deface the visible page content or silently capture and exfiltrate sensitive information entered into the page^[2].

Several real world incidents demonstrates how serious the impact can be. An XSS vulnerability in Yahoo Mail was exploited to hijack user accounts, a stored XSS vulnerability in Paypal's payment service was used to inject additional malicious code, another issue in Cisco's ASA VPN portal was also used to steal user credentials. A more serious example would be a certain zero day that exists in even in the latest version of the UltraViewer client. It has a (reflected) XSS vulnerability that lets you perform remote code execution on victim computers because it's an electron app and it has been set up in a way allowing attackers to easily bypass its sandboxing measures.

Since the malicious script executes within a trusted origin, browser protections like same-origin policies and other stuff do not block it in the same way they would block scripts from a separate website. This is one of the main reasons XSS continues to affect both smaller web apps and large, well resourced platforms.

Mitigation

Just one mechanism doesn't fully prevent XSS attacks. Multiple mitigation techniques should be used together to fully mitigate XSS attacks from happening.

- Output encoding: any value input by the user should be encoded accordingly (according to where its being inserted to, whether it's the DOM, a URL parameter or JS). This will ensure that special characters like <, >, /, etc... will be represented as normal text (after being encoded) rather than being interpreted by the browser like executable mark up.
- Input validation: apps should restrict and set the expected format / type / length / bounds / etc.. for any user input value. Apps should never trust the user. Even if we were to have this, this should be

combined with other mechanisms as well. This alone is not a preventive measure to stop XSS. This is because in certain circumstances, there might be certain inputs which could potentially include dangerous characters.

- Safe DOM APIs: client side JS should use safer properties like `textContent` when inserting untrusted data into the page. Unsafe sinks like `innerHTML` and `document.write` should be avoided unless the content has been properly handled – because they don't provide any encoding/sanitization mechanisms by default.
- CSP: have a properly restrictive CSP which limits the scripts that the browser is allowed to execute. It can block inline scripts and scripts loaded from authorized sources, reducing the impact of an injection even when another security control fails. This gives an extra defense in depth layer.
- Secure cookie attributes: session cookies should be set with the `HttpOnly` attribute to prevent client side JS from directly reading them. It doesn't prevent XSS directly, but it will reduce the attack surface by lowering the amount of information available to a malicious payload. It will help prevent direct session cookie theft in case if a injected successfully executes.
- Framework level protections: modern templating engines and web frameworks have features to auto escape values before they are injected to the DOM if used properly. For example, with JSP, you have the `<c:out>` for XML and HTML escaping, and many MVC frameworks also happen to have it enabled automatically by default. It's the same with a lot of industry standard JS frameworks like NextJS as well. These protections will reduce the chance of devs accidentally inserting untrusted data into the page – but again, you should be careful when dealing with non HTML context even with frameworks like these.

Conclusion

XSS still remains one of the most common vulnerabilities, not only on web apps but also on new desktop apps due to the increased trend in using electron and web technologies like JS where they are not supposed to be used. Different types of XSS differ in where the malicious payload is stored and how it reaches the victims browser but the underlying concept is the same for all of them. In each case, its untrusted user input being treated as legitimate HTML or executable code.

The most effective way to reduce the risk is to apply context aware output encoding consistently, use safe client side development practices, and use all of the other security practices mentioned above^{[1][3]}.

References

- [1] Mozilla, "URLSearchParams: Get() Method - Web APIs | MDN," Mozilla.org, Mar. 06, 2024. https://developer.mozilla.org/en-US/docs/Web/API/URLSearchParams/get#browser_compatibility (accessed July 28, 2026).
- [2] B. B. Gupta and P. Chaudhary, *Cross-Site Scripting Attacks: Classification, Attack, and Countermeasures*. Boca Raton, FL, USA: CRC Press, 2020.
- [3] MITRE, "Capec - Capec-63: Cross-Site Scripting (XSS) (Version 3.5)." Accessed: July 28, 2026. [Online]. Available: <https://capec.mitre.org/data/definitions/63.html>
- [4] OWASP, "Cross Site Scripting (XSS) | Owasp," OWASP, 2020. Accessed: July 28, 2026. [Online]. Available: <https://owasp.org/www-community/attacks/xss/>
- [5] MITRE, "CWE - CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-Site Scripting') (4.1)," July 2006. Accessed: July 28, 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/79.html>

[6] S. Lekies, B. Stock, and M. Johns, "25 million flows later: large-scale detection of DOM-based XSS," in **Proc. 2013 ACM SIGSAC Conf. on Computer and Communications Security**, Berlin, Germany, 2013, pp. 1193–1204.